Yegor Brusnyak

Messaging Bot Developer

Bratislava, Slovakia · English-first

TELEGRAM · DISCORD · SLACK · WHATSAPP-STYLE WEBHOOKS

Bot systems with channel adapters, dialogue state, fallbacks, logs, and human review.

I build messaging bots and bot backends around the parts that matter in real use: platform-specific payloads, webhooks, conversation state, approval queues, LLM fallback behavior, and operational visibility.

This page is targeted at bot-development roles such as WhatsApp Business API, Telegram Bot API, Discord API, Slack, LLM workflow, and webhook automation work.

4	2	5	1
channel families shown	live bot examples	runtime smoke suites	shared state graph

Demo Recording

Bot Proof

Primary proof · Node runtime

Channel Agent Runtime

Config-driven multi-channel runtime where Telegram, Discord, Slack, and WhatsApp-style events normalize into one LangGraph state graph for route selection, tool execution, JSONL audit, and approval-first replies.

GitHub **Why it fits bot roles**

Telegram Bot API · Booking flow

Telegram Salon Bot

Appointment-setting Telegram bot with inline keyboard flow, stylist/service/date/time selection, SQLite-backed bookings, admin commands, reminders, and optional operations webhook events.

GitHub **Demo video**

Discord API · Moderation

Discord Moderation Bot

Discord bot with slash commands, warning ladder, tickets, auto-moderation, audit logging, SQLite case history, safe demo incidents, and optional LLM moderation assistance.

GitHub **Demo video**

Webhook lab · Operator workflow

Messaging Bot Ops Lab

Local bot operations system with provider-shaped webhook endpoints, persisted conversation state, raw payload visibility, intent/urgency extraction, eval checks, and approval/rejection endpoints before outbound replies.

Code path **Production boundary**

Supporting proof · Bot-connected product

Lingonberry Journal

Trading journal product with Telegram bot commands, web dashboard, analytics, TradeLocker integration, and Telegram Mini App style workflow. Useful proof that bot actions can connect to real stored workflow state, not just chat demos.

GitHub **Supporting workflow proof**

Role Fit

Messaging platforms

- Telegram via Bot API and grammY/python-telegram-bot.
- Discord via discord.js and discord.py with commands and events.
- Slack via Bolt Socket Mode in the shared runtime.
- WhatsApp-style inbound webhooks for Meta/Twilio/Hermes-shaped payloads.

Reliability layer

- Conversation state persisted for review and replay.
- Human approval before risky outbound replies.
- Raw payload inspection for malformed provider events.
- Template fallback when LLM drafting fails.

Backend integration

- REST/webhook endpoints for Telegram, Discord, and WhatsApp-shaped events.
- SQLite/JSONL persistence for conversation state, bookings, and audit history.
- Tools for availability checks, route decisions, moderation actions, and handoff.

AI-assisted code review fit

- Bot logic is testable through smoke suites and deterministic simulations.
- AI-generated output is treated as draft behavior, not as trusted infrastructure.
- Failures are surfaced through logs, evals, and review gates instead of silent sends.

Honest Boundaries

What is live, tested, and simulated

Telegram and Discord proof are live-testable bot examples. Slack auth and send checks are validated in the runtime. WhatsApp work is currently webhook handling and local workflow proof; production sending requires Meta Cloud API or Twilio sender setup, templates, opt-in rules, and account credentials. The demos are portfolio proof and implementation evidence, not inflated enterprise client deployments.

Best fit: messaging bots, webhook automation, LLM bot reliability, and operator review workflows.

Available for remote freelance, part-time, or contract bot-development work across Telegram, Discord, Slack, and WhatsApp-style systems.

brusnyakyegor@gmail.com, GitHub, LinkedIn, General CV